

Manual técnico

Introducción

C++Disk 2.0 es la evolución del sistema de archivos simulado del Proyecto 1, diseñado para ofrecer una interfaz gráfica web intuitiva que permite la administración, visualización y gestión de discos, particiones y estructuras EXT2/EXT3. El sistema separa la lógica de bajo nivel (backend en C++) de la capa de presentación (frontend web), desplegado íntegramente en AWS mediante EC2 (backend) y S3 (frontend). El proyecto simula la creación, formateo, montaje y manipulación de un sistema de archivos real mediante archivos binarios .mia, incorporando journaling, control de permisos POSIX y comandos avanzados de gestión.

Arquitectura del Sistema

Frontend

Aplicación web estática desarrollada con [React/Angular/Vue]. Alojada en un bucket S3 de AWS configurado como Static Website Hosting. Se encarga de la GUI, terminal interactiva, visualizador de archivos y renderizado del journaling.

Backend

Servicio RESTful escrito en C++17/20, compilado con g++ o cmake. Ejecutado en una instancia EC2 con Ubuntu Linux. Gestiona la lógica de discos, particiones, EXT2/EXT3, permisos, usuarios y journaling.

Comunicación

HTTP/JSON mediante endpoints REST. CORS habilitado para permitir peticiones desde el dominio S3. Los comandos se envían como strings o JSON y se retornan respuestas estructuradas o mensajes de error.

Almacenamiento

Los discos se simulan en archivos .mia almacenados en el sistema de archivos de la EC2. La estructura interna sigue el modelo EXT2/EXT3 con Superblock, Bitmaps, Inodos y Bloques de datos.

Estructura de Datos:

MBR

Master Boot Record. Contiene la tabla de particiones primarias/extendidas (4 entradas), firma 0x55AA y tamaño del disco.

EBR

Extended Boot Record. Encadenado en lista para particiones lógicas. Contiene puntero a la siguiente EBR o NULL.

Superblock

Metadata del sistema de archivos: número de inodos, bloques, tamaño de bloque, estado del FS, tipo (EXT2/EXT3).

Bitmap de Inodos/Bloques

Vector de bits donde 1 = ocupado, 0 = libre. Permite asignación eficiente según fit (BF, FF, WF).

Tabla de Inodos

Estructuras que apuntan a bloques de datos. Almacenan permisos, propietario, tamaño, timestamps y punteros a bloques.

Bloques de Datos

Espacio físico para contenido de archivos y metadatos de directorios (estructuras FolderBlock y FileBlock).

Journaling (EXT3)

Bitácora que registra operaciones antes de ejecutarlas. Estructura: Operación, Ruta, Contenido, Timestamp. Tamaño fijo: 50 entradas constantes.

Comandos Implementados y pequeña Funcionalidad

Comando	Categoría	Parámetros Clave	Funcionalidad
fdisk	Particiones	-size, -unit, -path, -type, -fit, -add, -delete, -name	Crea, elimina o redimensiona particiones. Validación de restricciones MBR/EBR.
mount	Montaje	-path, -name, -id	Asigna un ID correlativo y monta la partición en memoria.
unmount	Desmontaje	-id	Desmonta partición, reinicia correlativo a 0 y libera recursos.
mkfs	Formateo	-id, -type, -fs	Formatea en EXT2 (2fs) o EXT3 (3fs). Inicializa Superblock, Bitmaps e Inodos.
mkusr/mkgrp	Usuarios	-name, -password, -group	Crea usuarios/grupos en la partición montada.
login/logout	Sesión	-user, -password, -id	Autentica usuario, establece contexto de

			permisos y sesión activa.
mkdir/mkfile	Creación	-path, -name, -content (mkfile)	Crea directorios o archivos con permisos por defecto 664.
remove	Eliminación	-path	Elimina archivo/carpeta recursivamente si hay permisos de escritura.
rename	Renombrado	-path, -name	Cambia nombre validando permisos y colisiones.
copy/move	Copiar/Mover	-path, -destino	Copia/mueve recursivamente. move solo actualiza punteros si es misma partición.
find	Búsqueda	-path, -name (? , *)	Búsqueda recursiva con patrones glob.
chown	Propiedad	-path, -usuario, -r	Cambia dueño recursivo. Solo root o dueño original.
chmod	Permisos	-path, -ugo, -r	Cambia permisos en octal (0-7). Solo dueño o root.
loss	Simulación	-id	Limpia Bitmaps, Inodos y Bloques con \0 para simular corrupción.

journaling	Auditoría	-id	Muestra bitácora en la GUI: operación, ruta, contenido, fecha/hora.
------------	-----------	-----	---

El Despliegue

Backend:

Se despliega en EC2 utilizando una IP publica generada por EC2.

Luego de eso se configura para que reciba respuestas de TCP para que pueda correr en S3 o buckets.

De esta manera usando CROW se manda las peticiones y se reciben para que puedan responder usando la API.

Se muestra las salidas de los comandos además de mostrar el inicio de sesión y todos los Discos, Particiones, Archivos y Carpetas creadas por el usuario usando un .smia o escribiendo comando por comando.

Frontend:

Se despliega en S3 un Bucket que tiene nuestro Frontend creado en React usando TypeScript, se crea la carpeta /dist usando npm run build.

Se sube a la nube de S3 esta carpeta usando index.html y el IP pública del EC2 para que pueda funcionar la API.

Esta muestra la terminal y todos los botones como Iniciar sesión y Journalist que muestra el journalist de los comando que hemos hecho.

Conclusión

El desarrollo de C++Disk 2.0 ha representado un hito significativo en la consolidación de competencias técnicas relacionadas con la implementación de sistemas de archivos, la programación de bajo nivel y la arquitectura de software distribuido. A través de este proyecto, se logró trascender la simulación básica de discos para construir una plataforma robusta que integra la lógica interna de sistemas EXT2 y EXT3 con una interfaz web moderna desplegada en la nube.

La implementación exitosa de estructuras críticas como el Superblock, los Bitmaps, la tabla de Inodos y, especialmente, el sistema de Journaling, permitió comprender la complejidad detrás de la integridad de datos y la recuperación ante fallos (loss). La gestión de permisos POSIX mediante comandos como `chmod` y `chown`, junto con operaciones recursivas (`copy`, `move`, `remove`), reforzó la importancia de la seguridad y el control de acceso en entornos multiusuario.

Desde una perspectiva de infraestructura, el despliegue en AWS (EC2 + S3) demostró la viabilidad de separar responsabilidades entre el backend y el frontend garantizando escalabilidad y accesibilidad. La comunicación vía APIs RESTful no solo optimizó el rendimiento, sino que permitió una experiencia de usuario fluida, donde la visualización gráfica del sistema de archivos complementa la potencia de la terminal de comandos.